Exercise 9 — Neural Networks

Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

Set 20 November 2026 · due Friday 27 November 2026, 09:00, at the start of lesson 10

Contents

What this exercise is for

Lesson 9 put two results side by side on purpose. On Meridian's acceptance data a linear model scored 0.55 and a network 0.94 — a hidden layer was worth 39 points. On the handwritten digits a linear model scored 0.93 and the best network on the table 0.97 — the same hidden layer was worth 2.8 points, and doubling its width after that was worth 0.09, which is a fifth of the seed-to-seed spread and therefore nothing.

Both numbers are in the handout, so both can be recalled. What cannot be recalled is which of *two datasets you have not seen* is which. That is what this exercise asks.

You get **two batches from the same factory**, Meridian Instruments, one from each of its two final-test stations. Both batches have 3,000 units, both have eight measured columns, both have a pass rate near 0.50, and on both the test station records 3% of its verdicts as the opposite of the truth — so both have a ceiling of 0.97, and every accuracy below is read against 0.97 rather than against 1.000.

Everything that could explain a difference has been held fixed except one thing: **the shape of the rule that decides the verdict**. On one line a hidden layer is the difference between a model that works and a model that does not. On the other it buys nothing measurable, and the multilayer perceptron (MLP) you fit will most likely come out *slightly below* logistic regression rather than above it.

Neither ordering can be guessed from the names of the two models, because the two models are the same on both lines. Both can be predicted from twenty minutes spent reading the generator's

docstring and plotting the columns. That is the skill being assessed — not fitting a network, but **looking at a problem and knowing in advance whether depth is what it needs.**

Submit a single notebook, `exercise09_<surname>.ipynb`, that runs top to bottom in the course container in under five minutes. Written answers go in markdown cells beside the evidence for them.

The two production lines

`production_line_data.py` ships with this exercise and needs no network, no download and no key.

```
from production_line_data import (
    load_lens_assembly, load_burn_in_screen,
    LENS_FEATURES, BURN_IN_FEATURES,
    TRUE_RIG_ERROR_RATE, TRUE_BAYES_ACCURACY,
)

lens = load_lens_assembly()     # optical bench line: a cell pressed into a barrel
burn = load_burn_in_screen()    # laser-diode line: modules screened for early failure
```

Each loader returns one DataFrame: eight measured columns, the **recorded** verdict, and a noise-free `truly_*` column.

	measurements	recorded verdict	noise-free verdict
lens assembly	`LENS_FEATURES` (8)	`assembly_passes`	`truly_within_tolerance`
burn-in screen	`BURN_IN_FEATURES` (8)	`survives_burn_in`	`truly_below_limit`

The `truly_*` column is for Part 4 and for nothing else. Never fit on it, never select on it, never score against it before Part 4. Meridian's own process engineers do not have it; it exists here only so that the station's 3% can be separated from a model's own mistakes, which no real batch record allows.

Read the module docstring in full before you do anything else. It describes both lines honestly and at length — what each product is, what is measured, and how each station's verdict is arrived at — and it is not padding. Half of Part 1 is answerable from that docstring alone.

Do not read past the loader functions until Part 4. The `TRUE_*` constants below them give the rules away.

Both loaders take `n_units`, `rig_error_rate` and `random_state`. Leave the first two at their defaults throughout, and vary only `random_state` where a part below asks you to.

Part 1 — Predict, before you fit anything (20 marks)

1. Load both batches and report, for each: the number of units, the number of measured columns, the recorded pass rate, and the majority-class baseline accuracy (the accuracy of always predicting the more common verdict). State the ceiling both lines share, and where the number comes from.

2. Look at the data, on both lines, **without fitting any model**. At minimum:

   - one plot per column showing its distribution split by the recorded verdict — a histogram pair, a box plot or a strip plot, whichever makes a shift easiest to see *if there is one*;
   - at least one two-dimensional scatter per line, of two columns against each other, coloured by the verdict. Choose which two by reading the docstring, not at random.

3. Then, in writing, **predict**: on which line will adding one hidden layer to a linear model buy a large improvement, and on which will it buy nothing? Commit to an answer for each line, and give a reason per line framed in the terms of lesson 9:

   - In your plots from step 2, does the verdict show up as a shift in individual columns — a passing unit tends to have a higher value here, a lower one there — or does no single column shift at all, so that the verdict only appears when two columns are looked at together?
   - From the docstring's description of each station (not from any `TRUE_*` constant): is the verdict described as a **weighted sum of the columns compared against a limit**, or as a condition on the columns **in combination**, which no weighted sum can express?
   - The handout's summary says a neuron is a line, and a hidden layer of H units is H lines with the output unit voting over them. For each line, how many straight boundaries would it take to separate the passing units from the failing ones, roughly? If the answer is one, what is a hidden layer for?

What is being marked here: the reasoning, not the prediction. A confident wrong prediction that engages with all three questions above scores well; an answer that hedges — "it might help on both, it depends" — scores badly even if it turns out to be defensible. Write the prediction into the notebook *before* the first `fit` call appears, and do not go back and edit it afterwards. The rest of the exercise is much less instructive if you do.

Part 2 — Measure it properly (25 marks)

4. For each line, build three models and score all three with **the same** 5-fold stratified cross-validation, reporting mean accuracy and the standard deviation across folds:

   - the majority-class baseline (`DummyClassifier(strategy="most_frequent")`);
   - **logistic regression**, the linear model of lesson 4;
   - a network with **one hidden layer of 8 rectified linear unit (ReLU) units**.

   Use scikit-learn's `MLPClassifier` for the network. It is a multilayer perceptron with exactly the machinery lesson 9 derived: `activation="relu"` is the rectified linear unit, and its default solver is adaptive moment estimation (Adam), the optimiser of handout section 9.3 — `solver="sgd"` would give you plain stochastic gradient descent (SGD) instead. Keras is allowed if you prefer it, but it is slower here and buys you nothing; whichever you choose, use it on **both** lines so the comparison is fair.

   The eight columns on each line are measured in different physical units and on wildly different scales — micrometres, newtons, nanometres, milliamps. Put a `StandardScaler` in a `Pipeline` in front of every model, so that the scaler is fitted inside each fold and never sees the fold it is scoring. A comparison in which one model was handed scaled inputs and the other was not is not a comparison of the models.

5. One draw of a synthetic generator tells you nothing about whether an effect is real. Repeat step 4 at **two further generator seeds per line** — pass a different `random_state` to the loader, keeping everything else fixed — and report each model's accuracy as a mean and standard deviation across the three batches.

6. State plainly, line by line: **is the network distinguishably better than logistic regression, distinguishably worse, or not distinguishable at all?** Justify the word you chose using the spread you measured, not the means alone. Handout section 10.3 is the pattern: three regularisers there landed 0.7 to 1.0 points apart against a seed-to-seed spread of 0.1 to 0.7, and the honest conclusion was "enough to say they helped, nowhere near enough to rank them".

One warning, now that your Part 1 prediction is written down and not before. If your **lens assembly** logistic regression comes out at essentially the majority-class baseline — half a point above it, or below it, on a batch where half the units pass — that is not a bug in your code, not a convergence warning you should silence with more iterations, and not a scaling mistake. Check it at a second generator seed and then explain it in Part 3. Handout section 2.2 has a coefficient vector of $(0.0096,\ 0.0701)$ that is doing the same thing for the same reason.

What is being marked: that the scaler lives inside the cross-validation rather than outside it; that no model ever sees a `truly_*` column; that every accuracy is reported with a spread; and that the word in question 6 is supported by that spread.

Part 3 — Explain the inversion (30 marks)

The answer to "does a hidden layer help here?" is not the same on the two lines. Explain why, in six to ten sentences per line.

Support every claim with a measurement from your own run, not with an assertion. These suggestions are worth what they cost; you may find better ones, and a better one you thought of yourself is worth more than three of these.

- *For a claim that a line's verdict does or does not show up in individual columns.* Compute, for each of the sixteen columns, the area under the receiver operating characteristic curve (AUC) of that single column used alone as a score for the verdict. An AUC of 0.5 means the column on its own carries nothing. Report the eight values per line, and say which line has columns that are individually informative — then say why that is the opposite of what a first guess would suggest.

- *For a claim about what the hidden layer is constructing.* On the line where the network wins, build **one** new column by hand from two of the original eight — the docstring tells you which two and what to do with them — and fit plain logistic regression on that single engineered column alone. Report its cross-validated accuracy against the eight-column network's. If a one-feature linear model gets close to the network, you have found what the hidden units were spending their capacity on.

- *For a claim about how much capacity the problem needs.* Sweep the hidden layer width over 1, 2, 4, 8 and 32 units and add a two-layer `(32, 32)`, on both lines, at one fixed generator seed, and plot accuracy against width. Run **three different `random_state` values of the network at every width** and report the mean, the spread and the best of the three; handout

section 3.4 is the reason, and the next bullet is what you will find. Say where the curve stops rising on each line. On the acceptance data section 11.2 found the step from two units to three worth 20 points and everything from three to thirty-two worth 0.5 — does your sweep have a step of that kind, on either line, and at what width?

- *For a claim about capacity against findability.* At the narrowest widths on one of these lines, the three initialisations will not agree with each other, and the spread between them will be far larger than anything else in this exercise. Report the three individual accuracies at that width rather than only their mean. Handout section 3.4 measured the same thing on the drift data — two units suffice, and plain gradient descent found them in 4 runs of 20 — and named the distinction: **representable and findable are different properties.** Say which of the two your narrow network is short of, and how you can tell.
- *For a claim that more capacity is not free.* Your sweep will show widths at which accuracy goes **down**, on both lines, and the two-layer `(32, 32)` is the worst configuration on both. Report where it starts and give a reason per line — the reasons are not the same, and six of the eight columns on one of these lines have nothing to do with its verdict, which is a relevant fact for exactly one of them.

What is being marked: this part carries the most marks in the exercise, and the measurements are what separate an explanation from a plausible story. A paragraph that says "this line's rule is non-linear, so the network wins" is worth almost nothing on its own — it restates the outcome in different words. A paragraph that says "not one of this line's eight columns reaches an AUC further than *this* from 0.5, and yet a single column I built by hand out of two of them reaches *that*, which is why no weighted sum of the raw eight can work and why one hidden layer of two units is enough" is the answer, because every italicised word in it is a number you measured.

The width sweep is the most expensive thing in this exercise — six configurations, three initialisations, two lines, five folds, so 180 fits. It runs in a little over two minutes on the container's central processing unit (CPU), and everything else here is seconds. Nothing in this exercise needs a graphics card, and nothing in it should take you more than three hours.

Part 4 — Mark yourself against the rule that made the data (25 marks)

Now open `production_line_data.py` and read the `TRUE_*` constants and the two loader bodies.

7. `TRUE_CLEARANCE_TOL_UM` and `TRUE_FIT_FEATURES` give the lens line's rule exactly; `TRUE_HAZARD_WEIGHTS` and `TRUE_HAZARD_LIMIT` give the burn-in line's. For each line, write the rule out in one sentence and say whether your Part 1 prediction was right, wrong, or right for the wrong reason. Be specific about which of the three questions in Part 1.3 you answered correctly and which you did not.

8. The lens rule is a condition on $|a - b|$ for two of the columns. Handout section 3.1 shows that the two-channel drift data's rule is a condition on $|a + b|$ — the exclusive-or (XOR) function, the smallest problem that needs a hidden layer — and section 3.2 solves it by hand with **two** ReLU units and no training at all. Write down, by hand, weights $W^{[1]}$, $b^{[1]}$, $W^{[2]}$, $b^{[2]}$ for a two-unit hidden layer that implements the lens rule on the two relevant standardised columns, in the style of that section. Then evaluate your hand-built network on the whole batch and report its accuracy against `assembly_passes`. You are not asked to train anything here.

9. Score your best model on each line against the `truly_*` column instead of the recorded verdict, and report both numbers. Then check the identity of handout section 11.3: if a model agrees with the true rule on a fraction q of units and the station records the wrong verdict with probability $e = 0.03$, its accuracy against the recorded verdict should be $q(1-e) + (1-q)e$. Report the predicted and observed values and the gap between them, per line. State in one sentence what this means for the burn-in line: how much of that line's remaining shortfall below 1.000 is the model's, and how much is the station's?

10. **The question this exercise exists for.** A new Meridian line is being commissioned. You are handed its specification document and one batch of measurements, and asked whether the model should be linear or should have a hidden layer — before any budget is spent. Using both lines here as evidence, name the **two** checks you would run first, and for each one say what result would push you towards depth and what result would push you away. At least one of your two checks must be something you actually computed in Part 3, quoted with its number on both lines.

What is being marked: question 10, and question 8 after it. Being able to look at a rule and say why one hidden layer is or is not the thing it needs — rather than treating "networks are better" or "start simple" as a rule that holds everywhere — is the whole content of this lesson.

Marking

Part	Marks
1 — Predict before fitting	20
2 — Measure it properly	25
3 — Explain the inversion	30
4 — Mark yourself against the rule	25
Total	**100**

Marks are lost for:

- fitting, tuning or selecting anything using a `truly_*` column before Part 4 (**−15**);
- standardising the whole dataset once, before the cross-validation split, instead of inside a `Pipeline` (**−10**);
- reporting a single run's accuracy where Part 2 asks for a mean and spread across three generator seeds (**−10**);
- comparing a network against logistic regression when only one of the two was given scaled inputs (**−10**);
- asserting a claim in Part 3 with no supporting number from your own run (**−5** per unsupported claim);
- a notebook that does not run top to bottom in the container (**−10**).

Marks are **not** lost for:

- a wrong prediction in Part 1, provided it engaged with all three questions. A wrong prediction you then investigated is worth more than a cautious one that committed to nothing;
- reporting that the network and logistic regression are **not distinguishable** on a line, and declining to name a winner. If your spreads genuinely overlap, that is the correct answer and

it earns full marks in question 6 — naming a winner anyway, on a difference smaller than the spread you measured, loses them. This is lesson 5's discipline and lesson 9's section 10.3 applied to your own numbers, and it is the habit that carries into the final project;
- a network that comes out **below** logistic regression, if you report it as measured and explain it rather than retuning until it wins;
- measuring the lens line's logistic regression at the majority-class baseline, provided you checked it at a second seed rather than assuming a bug.

Getting help

Email fabio.antonini.1969@gmail.com. There is no lesson between the day this is set and the morning it is due, so the inbox is the only channel.

If you are stuck on Part 1, handout section 2.2 does exactly this kind of reasoning for the acceptance data: it argues from the *symmetry* of the accept region that no line can help, and only then fits one to confirm it. If you are stuck on Part 3, section 11.2's width sweep is the template for the plot, and section 6.3's digits table is the template for the case where the honest answer is "the hidden layer bought 2.8 points and the second one bought nothing". If you are stuck on Part 4 question 8, section 3.2 is the worked example, four columns of arithmetic wide; the only thing you need to change is a sign.